



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS
COMPUTACIONALES
DEPARTAMENTO DE INGENIERÍA DE SOFTWARE
LICENCIATURA DE INGENIERÍA DE SOFTWARE
CURSO DE INGENIERÍA WEB
LABORATORIO GUIADO N°3



Profesora: Dra. Elba Valderrama Bahamóndez
2026

Grupo: 1SF-133

II SEMESTRE

A. **TEMA:** Arreglos, Funciones y Objetos en JavaScript

B. OBJETIVOS DE APRENDIZAJE:

1. Manipular arreglos usando métodos de orden superior (map, filter, reduce, forEach, find, sort).
2. Diseñar funciones reutilizables que reciban y devuelvan arreglos y objetos.
3. Modelar información del mundo real usando objetos y arreglos de objetos.
4. Combinar arreglos, funciones y objetos para resolver un problema

C. INSTRUCCIONES:

1. Resuelve los ejercicios en orden; cada parte se apoya en la anterior.
2. Usa el archivo ejercicios.js como punto de partida: contiene los TODO que debes completar.
3. Verifica cada ejercicio imprimiendo resultados con console.log antes de pasar al siguiente.
4. No modifiques los nombres de las funciones ni sus parámetros: se usa como referencia para revisar tu trabajo.

D. PROCEDIMIENTO:

Parte 1 — Arreglos

1.1 Calentamiento: acceso y modificación. Dado el siguiente arreglo:

```
const inventario = ["teclado", "mouse", "monitor",  
  "audífonos"];
```

- a. Escribe una expresión que obtenga el último elemento sin usar el índice numérico fijo (pista: .length).
- b. Agrega "webcam" al final y "cable HDMI" al inicio, sin usar .push/.unshift directamente en la consola: escribe una función agregarItems(arr, alInicio, alFinal) que devuelva un nuevo arreglo con ambos cambios, sin mutar el original.

1.2 Filtrado y transformación. Con este arreglo de temperaturas (en Celsius) registradas durante una semana:

```
const temperaturas = [18, 22, 25, 30, 15, 19, 27];
```

Implementa:

- aFahrenheit(arr) → devuelve un nuevo arreglo con las temperaturas convertidas a Fahrenheit usando map.

- díasCalurosos(arr, umbral) → devuelve solo los valores mayores al umbral usando filter.
- promedio(arr) → devuelve el promedio usando reduce.
- temperaturaMaxima(arr) → devuelve el valor máximo (puedes usar reduce o Math.max(...arr)).

1.3 Búsqueda y orden

- buscarPrimerMayorA(arr, valor) → usa find para devolver el primer elemento mayor que valor, o undefined si no existe.
- ordenarDescendente(arr) → devuelve una **copia** ordenada de mayor a menor (cuidado: .sort() muta el arreglo original; usa el operador de propagación [...arr] para copiarlo primero).

Revisión 1: Antes de continuar, muestra en consola el resultado de cada función con temperaturas como entrada.

PARTE 2 - FUNCIONES

2.1 Funciones como parámetros (callbacks). Implementa una función genérica de recorrido:

```
function procesarLista(arr, accion) {
  // TODO: recorre arr y aplica "accion" a cada elemento, devolviendo un nuevo arreglo
}
```

Úsala para:

- Duplicar cada número de un arreglo.
- Convertir cada palabra de un arreglo de strings a mayúsculas.

Reflexiona: ¿en qué se parece procesarLista a .map? ¿Por qué el lenguaje ya trae ese método incorporado?

2.2 Funciones flecha y funciones de orden superior

Reescribe usando arrow functions una función crearMultiplicador(factor) que devuelva otra función capaz de multiplicar cualquier número por factor. Ejemplo de uso:

```
const porTres = crearMultiplicador(3);
porTres(10); // 30
```

Explica en un comentario qué es un closure y dónde aparece en este ejercicio.

2.3 Validación y manejo de errores

Implementa dividirSeguro(a, b) que:

- Devuelva el resultado de a / b si b !== 0.
- Lance un error (throw new Error(...)) con un mensaje claro si b === 0.
- Se use con try/catch en un ejemplo que muestre ambos casos (éxito y error).

Revisión 2: Muestra en consola al menos un caso exitoso y un caso de error capturado con try/catch.

Parte 3 — Objetos

3.1 Modelando una entidad

Crea un objeto producto con al menos estas propiedades: nombre, precio, stock y un método aplicarDescuento(porcentaje) que devuelva el precio con descuento sin modificar el precio original.

```
javascript const producto
= { nombre: "Teclado
mecánico",
  precio: 45,
  stock: 12,
  aplicarDescue
nto(porcentaje
) {
  // TODO
}
};
```

3.2 Arreglos de objetos

Trabaja con este catálogo:

```
javascript
const catalogo
= [
  { nombre: "Teclado", precio: 45, categoria: "periféricos", stock: 12 },
  { nombre: "Monitor", precio: 180, categoria: "pantallas", stock: 5 },
  { nombre: "Mouse", precio: 20, categoria: "periféricos",
stock: 30 }, { nombre: "Silla", precio: 150, categoria:
"mobiliario", stock: 0 },
];
```

Implementa:

- productosDisponibles(catalogo) → usando filter, devuelve solo los productos con stock > 0.
- nombresPorCategoria(catalogo, categoria) → usando filter + map, devuelve un arreglo con solo los nombres de los productos de esa categoría.
- valorTotalInventario(catalogo) → usando reduce, devuelve la suma de precio * stock de todos los productos.
- productoMasCaro(catalogo) → devuelve el objeto completo del producto con mayor precio.

3.3 Desestructuración (destructuring)

Reescribe nombresPorCategoria usando ****desestructuración**** en los parámetros de la función flecha del .map/.filter, por ejemplo:

```
javascript
catalogo.filter(({ categoria: cat }) => cat === categoria)
```

Comenta por qué la desestructuración hace el código más legible.

Parte 4 — Problema integrado

Una tienda quiere un pequeño reporte de ventas. Se te da:

```
javascript
const ventas
= [
  { producto: "Teclado", cantidad: 3, precioUnitario: 45 },
  { producto: "Monitor", cantidad: 1, precioUnitario: 180 },
  { producto: "Mouse", cantidad: 5, precioUnitario: 20 },
  { producto: "Teclado", cantidad: 2, precioUnitario: 45 },
  { producto: "Silla", cantidad: 1, precioUnitario: 150 },
];
```

Construye una función generarReporte(ventas) que devuelva un **objeto** con esta forma:

```
{
  totalVendido: 685,      // suma de cantidad * precioUnitario de todas las ventas
  numeroTransacciones: 5,
  productoTopVentas: "Teclado", // el que más "cantidad" total acumuló
  resumenPorProducto: [
    { producto: "Teclado", cantidadTotal: 5, ingresoTotal: 225 },
    { producto: "Monitor", cantidadTotal: 1, ingresoTotal: 180 },
    // ...
  ]
}
```

Pistas:

1. Usa reduce para agrupar las ventas por producto (puedes usar un objeto acumulador como "diccionario").
2. Convierte ese objeto agrupado en un arreglo con Object.values(...).
3. Usa sort sobre una copia del resumen para encontrar el producto con más ventas.

Al terminar, imprime el reporte completo con console.log(JSON.stringify(reporte, null, 2)).

Preguntas de cierre (para entregar en comentarios o al final del archivo)

1. ¿Qué diferencia hay entre map y forEach? ¿Cuándo usarías cada uno?
map: Transforma los elementos y devuelve un nuevo arreglo sin modificar el original. Úsalo cuando necesites obtener una lista transformada.
forEach: Ejecuta una función por cada elemento pero devuelve undefined. Úsalo solo para acciones secundarias (como imprimir en consola o modificar variables externas).
2. ¿Por qué reduce se considera el método "más general" entre los métodos de arreglos vistos?

Porque permite condensar un arreglo en cualquier tipo de dato (un número total, un objeto, un string o incluso otro arreglo). Es tan versátil que otros métodos como map o filter pueden recrearse usando solo reduce.

3. Da un ejemplo real (fuera de este laboratorio) donde modelarías datos como un arreglo de objetos.

Un carrito de compras de una tienda online: el carrito es el arreglo y cada elemento es un objeto que representa un producto con datos como id, nombre, precio, cantidad y talla.

4. ¿Qué ventaja tiene evitar mutar arreglos y objetos directamente?

Garantiza un código predecible y fácil de depurar, previniendo fallos inesperados al evitar que una función modifique datos que se usan en otra parte. Además, es esencial en la programación funcional y en frameworks como React para manejar estados de forma limpia.

E. ENTREGABLES:

1. En el espacio en comentario el link al git con el(los) archivo(s) javascript
2. La respuesta a las 4 preguntas

F. EVALUACION

- | | |
|---|---|
| 1. Respuesta a las preguntas | 4 |
| 2. Entrega Archivo JS | 2 |
| 3. Comprobación Individual Oral o Escrita | 4 |